Ingeniería Investigación y Tecnología
Volume XXVII (Issue 1), January-March 2026 1-12
ISSN 2594-0732 FI-UNAM Peer-reviewed article
Article information: Received: July 10, 2025, accepted: October 23, 2025.

https://doi.org/10.22201/fi.25940732e.2026.27.1.004

Ingeniería
Investigación y
Tecnología

Creation of discrete event simulation models using artificial intelligence and flexsim

Romero-Guerrero, Jorge Adan (Corresponding author)
CIATEQ. Manufactura Virtual Y Lean Y Cad Cae, México
E-mail: adan.romero@ciateq.mx
https://orcid.org/0000-0003-0431-1917

Bautista-Orduna, Guillermo Egberto
CIATEQ. Manufactura Virtual Y Lean Y Cad Cae, México
E-mail: guillermo.bautista@ciateq.mx
https://orcid.org/0009-0004-9308-6604

Suarez-Luna, Johovani Misael
UPP. Ciencias y Tecnologías Avanzadas, México
E-mail: johovanisuarez@micorreo.upp.edu.mx
https://orcid.org/0009-0009-4933-0346

Arenas-Islas, David
CIATEQ. Manufactura Virtual Y Lean Y Cad Cae, México
E-mail: david.arenas@ciateq.mx
https://orcid.org/0000-0001-6169-2045

Abstract

Discrete event simulation (DES) is a critical tool for decision-making in complex systems, but the creation of simulation models often demands substantial manual effort. This study investigates the potential of artificial intelligence (AI), specifically ChatGPT, to automate the generation of simulation models from industrial layout images. By leveraging natural language processing (NLP), FlexSim code is generated to accurately position machines within the simulation environment, significantly reducing the time and effort required for manual configuration. Through iterative interaction, the tool adapts to user-specific requirements, refining the generated code to improve accuracy and relevance. Experiments demonstrate the capabilities and limitations of using free AI tools for automating DES model creation, achieving a significant reduction in modeling time while maintaining high accuracy. While this approach complements rather than replaces traditional methods, it provides a scalable and efficient alternative for complex layouts. This work contributes to the field of AI-assisted engineering by showcasing the feasibility of integrating large language models into simulation workflows. By reducing barriers to entry and enhancing efficiency, this approach paves the way for more accessible and adaptive simulation tools, enabling industries to optimize processes and improve decision-making in dynamic environments.
Keywords: Discrete event simulation, ChatGPT, FlexSim, code generation, artificial intelligence, industrial layouts.

https://doi.org/10.22201/fi.25940732e.2026.27.1.004

Introduction

Discrete Event Simulation (DES) is a modeling technique that represents complex systems as a sequence of discrete events occurring at specific times, with each event triggering a change in the system's state. This approach enables the analysis of system behavior under diverse scenarios without disrupting real-world operations (Garrido, 2009). DES is widely applied across industries, such as manufacturing (e.g., optimizing production lines to reduce bottlenecks), logistics (e.g., enhancing inventory management and transportation efficiency), healthcare (e.g., improving patient flow for better resource allocation), and transportation (e.g., modeling traffic systems to minimize congestion) (García & Romero, 2020). By delivering insights into key metrics like throughput, waiting times, and resource utilization, DES facilitates data-driven decision-making and system optimization in dynamic environments.

FlexSim, a leading DES platform, enhances these capabilities with its intuitive 3D modeling and robust analytical tools. For example, in logistics, FlexSim models warehouse operations with autonomous guided vehicles (AGVs), optimizing routes and reducing cycle times using real-time data and tools like opt Quest. In healthcare, FlexSim HC simulates patient flows in emergency departments, cutting wait times through dynamic prioritization logic. In transportation, it optimizes airport baggage handling systems, boosting throughput via scenario analysis with Experimenter. Flex Sim's visual interface and flexible modeling environment enable users to create detailed, industry-specific models, supporting efficient analysis and optimization of complex systems (Flexsim, 2024).

In recent years, the development of artificial intelligence (AI) tools has opened new possibilities for automating and simplifying complex processes across various fields, including discrete event simulation (OpenAI, 2024). One such tool is ChatGPT, an AI-based language model developed by OpenAI, which has proven capable of generating text, code, and coherent responses from user-provided inputs (Hamilton, 2024). This article explores the integration of ChatGPT in the creation of discrete event simulation models using industrial layouts as a starting point, specifically within the FlexSim environment, a widely used simulation platform in industry. Recent literature has explored how these models can be integrated into industrial and academic settings to optimize processes and enhance productivity (Gonzalo & Garrido, 2023).

The integration of AI into simulation has been explored in various fields, including medical education, where tools like virtual patients and personalized learning systems have demonstrated the potential of AI to enhance training and decision-making (Hamilton, 2024). In the industrial domain, AI can similarly transform simulation modeling by automating repetitive tasks, improving accuracy, and making the process more accessible to users with limited programming experience. Techniques such as natural language processing (NLP) and computer vision enable AI tools like ChatGPT to interpret user inputs, including textual descriptions and images, and generate functional simulation models. This capability creates new possibilities for creating adaptive and context-aware simulation environments, particularly in platforms like FlexSim.

Recent advances in multimodal pre-trained models, such as GPT-4 with vision capabilities, have demonstrated the ability to process and integrate multiple data modalities (e.g., text images) to generate richer and more contextualized outputs (Wang *et al.,* 2023). These models, including BERT, GPT, and ViT, provide a foundation for integrating AI into complex applications, such as industrial simulation. By leveraging multimodal capabilities, ChatGPT can interpret industrial layout images and generate corresponding simulation code, significantly reducing the manual effort required for model creation.

The opportunity addressed in this study is to reduce the time and expertise required to develop DES models, making them more accessible and scalable. Traditional DES model development often follows a structured methodology, such as the one proposed by Law and Kelton (Averill, 2015), which includes stages like problem formulation, data collection, model building, verification, validation, and experimentation. The model-building stage, which involves translating system specifications into a simulation environment like FlexSim, is particularly labor-intensive, often consuming 40-60 % of the total project time (depending on the layout and user expertise) (Banks, 2010). Manual configuration of elements, such as defining machine locations and connections, can take hours or days, especially for intricate industrial layouts.

However, complex models often require advanced programming skills and significant manual effort. Developing DES models in FlexSim relies on FlexScript and C++ across project stages, as outlined by Law and Kelton's methodology (Averill, 2015): Problem Formulation and Data Collection may use FlexScript for structuring inputs within FlexSim; Model Building employs FlexScript for creating objects,setting properties, and defining flows with C++ for custom logic via DLLs; Verification and Validation use FlexScript for debugging and testing; Experimentation/Optimization relies on FlexScript for scenario testing and C++ for advanced optimization algorithms through DLLs.

This study proposes the integration of artificial intelligence (AI), specifically ChatGPT, to automate the model-building stage of DES development. By leveraging natural language processing (NLP) and image recognition capabilities, ChatGPT interprets industrial layout images and generates FlexSim code to create simulation models with minimal manual intervention.

Recent advancements in AI have led to powerful tools like ChatGPT, which leverage neural networks to understand and generate text based on user input. ChatGPT's ability to generate coherent and contextually relevant outputs, including code, makes it a versatile tool for automating complex tasks such as simulation modeling (Herrera *et al.,* 2024). However, while ChatGPT can significantly reduce manual effort, its output may require validation and refinement to ensure accuracy and relevance. In this study, we explore how iterative interaction with ChatGPT can refine the generated code, progressively adapting to the user's specific needs and improving the simulation models' quality.

Despite the growing use of AI in simulation, there remains a lack of tools that can automate the creation of DES models from unstructured inputs, such as industrial layout images. This study addresses this gap by proposing a novel approach that integrates ChatGPT into the FlexSim environment, enabling simulation code generation from layout images with minimal manual intervention. By automating the initial model creation process, our approach allows users to focus on analysis and optimization, significantly reducing the time and expertise required to build complex simulation models. This work contributes to AI-assisted engineering by demonstrating the feasibility of using large language models to enhance simulation workflows.

Literature review

Integrating artificial intelligence (AI) tools, particularly large language models like ChatGPT, into simulation modeling is an emerging area of research with significant potential. Recent studies have explored the use of AI for tasks such as code generation, program repair, and simulation optimization, highlighting both the opportunities and challenges of these technologies. This literature review examines key studies in this area, focusing on the capabilities and limitations of ChatGPT for automating the creation of discrete event simulation (DES) models, particularly within the FlexSim environment.

AI-Assisted code generation

Several studies have explored the use of generative AI models for code generation, providing valuable insights into their capabilities and limitations. Megahed *et al.* (2023) demonstrate that ChatGPT can efficiently generate code and explain basic concepts in structured tasks, such as statistical process control (SPC). However, they also note that the model struggles with more complex tasks, such as creating code from scratch or explaining lesser-known concepts. Similarly, Tian *et al.* (2023) evaluated ChatGPT's effectiveness as a programming assistant, finding that while it excels at generating code for common problems, it struggles to generalize to new or unseen problems. These findings suggest that while ChatGPT can be a powerful tool for automating code generation, its outputs require careful validation and refinement.

The article by Hou & Fidopiastis (2016) provides a conceptual framework for intelligent adaptive learning systems, which may be relevant for integrating ChatGPT into simulation. The authors discuss how intelligent tutoring systems (ITS) can dynamically adapt to user needs, optimizing the learning process and facilitating knowledge transfer to operational environments.

One of the latest advancements in this field is the application of AI models in the automation of simulation modeling. Jackson *et al.* (2023) demonstrate that a framework based on generative models like GPT-3 Codex can translate natural language descriptions into functional simulation models. In their study, they successfully generated simulations of queueing systems and inventory management from textual descriptions, showing that AI can significantly reduce the burden of manual programming in the simulation of industrial systems.

In a related study, Sadik *et al.* (2024) propose an agile model-driven development (MDD) approach that uses GPT-4 to generate code from Unified Modeling Language (UML) diagrams. Their results demonstrate that incorporating semantic and ontological constraints into UML models leads to more complex yet manageable code generation. This approach highlights the potential of AI to automate software development tasks, including creating simulation models. However, the authors emphasize the importance of human oversight to ensure the accuracy and relevance of the generated code.

Limitations of ChatGPT

Despite its capabilities, ChatGPT has notable limitations that must be addressed when using it for simula-

tion modeling. Ma *et al.* (2023) conducted an extensive study on ChatGPT's ability to understand code syntax and semantics, finding that while the model understands syntax, it struggles with dynamic semantics. This suggests that while ChatGPT can generate initial code for FlexSim models, the outputs require manual validation and tuning to ensure they behave as expected in simulation. Similarly, Majeed & Hwang (2024) highlight challenges related to the accuracy and reliability of ChatGPT-generated results, particularly in applications where precision is critical, such as simulation. These studies underscore the importance of balancing automation with human oversight to ensure the quality and reliability of AI-generated models.

Ethical and practical considerations

Integrating ChatGPT into simulation modeling also raises important ethical and practical considerations. Combrink *et al.* (2023) provide practical recommendations for the ethical use of ChatGPT in academic settings, emphasizing the need for clear policies to ensure the originality of work and prevent plagiarism. These recommendations are particularly relevant in the simulation context, where AI-generated models must be transparent and accountable. Additionally, Kobiella *et al.* (2024) explore how the use of ChatGPT affects users' perceptions of productivity and accomplishment. While some users report increased efficiency, others express concerns about a diminished sense of ownership and achievement. These findings highlight the importance of designing AI-assisted tools that enhance, rather than replace, human creativity and engagement.

AI in industrial applications

The application of AI in industrial settings has demonstrated significant potential for improving efficiency and flexibility. Wan *et al.* (2021) discuss how AI technologies are transforming manufacturing processes, enabling the creation of smart factories that can dynamically adapt to customized production needs. These advancements are particularly relevant to the simulation of complex industrial systems, where AI can play a key role in optimizing processes and reducing manual effort. By leveraging AI tools like ChatGPT, simulation modeling can become more accessible and scalable, supporting the development of smarter and more adaptive industrial systems.

Recent studies have shown that generative AI models can significantly enhance productivity across various sectors. For example, Brynjolfsson *et al.* (2023) found that the use of AI tools in customer service centers increased productivity by 14 %, with particularly notable benefits for less experienced workers. These findings suggest that AI can play a significant role in training and accelerating learning in complex environments, such as discrete event simulation.

Historical context of AI

The history of AI development offers valuable lessons for integrating tools like ChatGPT into simulation modeling. Muthukrishnan *et al.* (2020) provide a historical perspective on the boom-and-bust cycles in AI, known as "AI winters," characterized by periods of high expectations followed by disappointment when technology fails to deliver on its promises. These cycles underscore the importance of maintaining realistic expectations and recognizing the limitations of AI tools. While ChatGPT represents a significant advancement, its application in simulation must be carefully managed to avoid overreliance and ensure sustainable progress.

Creating objects using code in FlexSim

The FlexSim scripting console executes Flexscript commands without running the entire model. This is useful for obtaining information or making configurations in the model. Type the code in the indicated field; if it has a return value, it will appear in the results field (Flexsim, 2024). Firstly, it is important to mention that the model created will be basic, serving as a simple entry into using FlexSim and its integration with artificial intelligence tools. According to Bilgram & Laarmann (2023), large language models (LLMs) such as ChatGPT can significantly accelerate the initial phases of innovation, from exploration and idea generation to digital prototyping, allowing non-technical users to generate code from natural language descriptions. This simple model will use fundamental elements such as Source, Processor, and Sink. These elements allow for establishing a basic flow within the simulation environment.

To learn the necessary commands, use the information on Coding in FlexSim, located on the official FlexSim website. In FlexSim, object instances are created using the createinstance command, although this function has been replaced by Object.create() in more recent versions (Flexsim, 2024). This change reflects FlexSim's evolution toward a more object-oriented approach, making creating and manipulating entities within the model easier.

To define the location of each element in the workspace, use the command:

Object.setLocation(double x, double y, double z), which allows you to specify the exact coordinates of the object in the model. To determine the size of the machine, use the command *Size.Array*(3).

Based on this information, below is an example of how to create a machine named "M-M1," located at the coordinates x: 27.50 m, y: 20 m, and z: 0 m, with dimensions of 2 meters on the x, y, and z axes. The code for this creation is shown in Figure 1.

Other commands exist for machine location:

Object obj = Model.find("Processor1");
obj.location = Vec3(0,0,0);

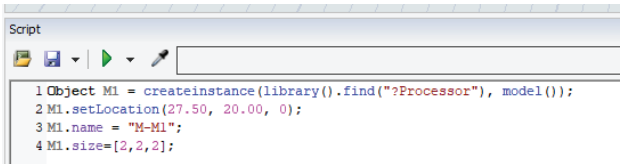

Figure 1. Code for creating a machine.

Figure 2 shows the result: a machine named M-M1 has been successfully created. This machine measures 2 meters on the x, y, and z axes and is located on the inner left side of the x, y plane at 27.5 meters in x and 20 meters in y. In Figure 2, different coordinates can be seen because the software takes the center of the geometry as a reference.

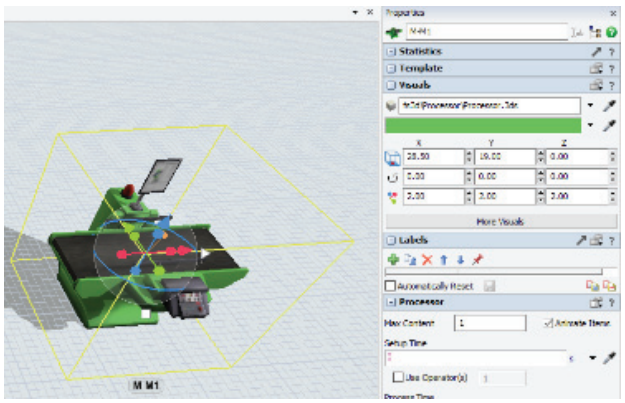

Figure 2. Creating the machine from the given code.

Creating connections between elements using code

Once the code for creating machines was validated, we verified that the same approach applies to creating buffers, sinks, and operators. The next step was to create connections between these elements. In FlexSim, there are three types of connectors:

a) Object Connector:

Used to connect objects or groups of objects according to the process flow. It is activated by clicking the icon and then the start and end objects.

b) Central Port Connector:

Like the object connector, it is designed to connect central ports of elements that move FlowItems (such as resources or machinery).

c) Extended Connection:

Allows for extended connections between objects according to the model, such as connecting a processor, dispatcher, and operators in a network of nodes. To connect objects, the following commands can be added to the code:

1. contextdragconnection (obj fromobject, obj toobject, str/num characterpressed)
2. objectconnect (obj object1, obj object2)

These commands connect two objects, adding the necessary ports. An output port is added to object1 and an input port to object2, as needed.

A test was performed connecting "Source_1" to "Machine_1" using the following code, shown in Figure 3.

objectconnect(Model.find("Fuente_1"),Model.find("Machine_1"))

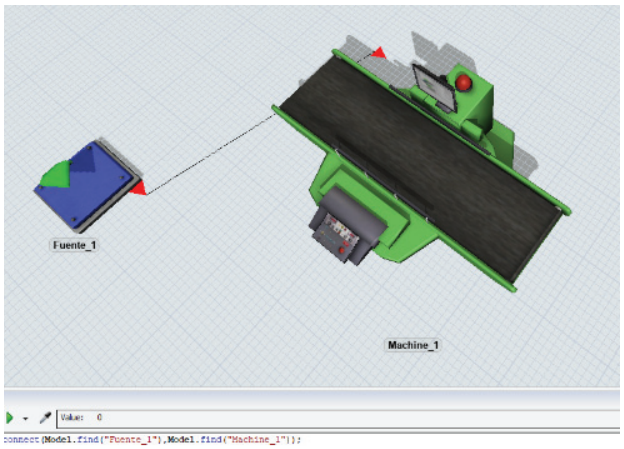

Figure 3. Connection between source and machine from the code.

https://doi.org/10.22201/fi.25940732e.2026.27.1.004

Layout interpretation using artificial intelligence

Once the code for creating, placing, and connecting elements was validated, an image was entered into ChatGPT to evaluate its recognition and code generation capabilities.

A basic sketch was designed with six elements: One source, three machines, and two sinks in a 10x5 meter space. Each element measures 1x1 meter, and the units in the image are expressed in millimeters. The sketch is shown in Figure 4.

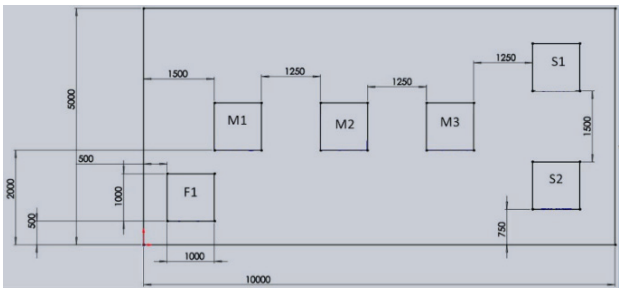

Figure 4. Layout introduced to ChatGPT.

Once the figure was attached in .jpg format, the following request was made:

"Using the reference image, extract information about the number of elements, their dimensions, and their locations on the x and y axes, using the lower left corner as the origin.". Table 1.

Table 1. Dimension and locations of the elements interpreted by ChatGPT in Figure 4.

Element	Dimensions	Location x (mm)	Location y (mm)
F1	1000x1000	500	500
M1	1000x1000	2000	3000
M2	1000x1000	4250	3000
M3	1000x1000	6550	3000
S1	1500x1500	8750	4500
S2	1500x1500	8750	750

Analyzing the data returned by ChatGPT, it is observed that the dimensions of the objects vary in Sinks 1 and 2, and only the correct location is obtained for the Source. The correct dimensions and location information is presented in Table 2.

Table 2. Dimensions and location of the elements indicated in Figure 4.

Element	Dimensions	Location x (mm)	Location y (mm)
F1	1000 × 1000	500	500
M1	1000 × 1000	1500	2000
M2	1000 × 1000	3750	2000
M3	1000 × 1000	6000	2000
S1	1000 × 1000	8250	3250
S2	1000 × 1000	8250	750

Several tests were performed, such as changing the background color, increasing the image resolution, and renaming the objects, but the results were consistent. Therefore, it was decided to add dimension and location dimensions for each element and indicate the connections between them. The output was indicated in the upper right corner and the input in the lower left corner, allowing the input and output of each element to be specified.

The layout became graphically saturated with information by including seven elements and four operators, as shown in Figure 5.

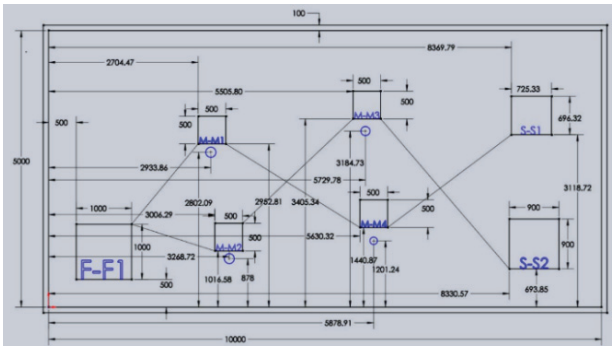

Figure 5. Layout with all referenced geometries and dimensions.

The data for each element, such as its dimensions, location, and the input and output values provided by ChatGPT, are shown in Table 3.

Table 3. Dimension and location of the elements interpreted by ChatGPT in Figure 5.

	Dimensions	Location x (mm)	Location y (mm)	In	Out
F1	1000x1000	500	500		M1
M1	500x500	3006.29	2933.86	F1	M4
M2	500x500	2802.09	1016.58		M3
M3	500x500	5505.80	3184.73	M2	M4
M4	500x500	5630.32	1440.87		S1
L1	-	3268.72	3268.72		
L2	-	3268.72	3268.72		
L3	-	5729.78	3405.34		
L4	-	3405.34	1440.87		
S1	1000x1000	8369.79	3118.72	M4	S2
S2	1000x1000	9078.91	693.85	S1	

The correct data is shown in Table 4.

Table 4. Dimensions and location of the elements indicated in Figure 5.

	Dimensions	Location x (mm)	Location y (mm)	In	Out
F1	1000x1000	500	500)		M1, M2
M1	500x500	3006.29	2933.86	F1	M4
M2	500x500	2802.09	1016.58	F1	M3
M3	500x500	5505.80	3184.73	M2	S2
M4	500x500	5630.32	1440.87	M3	S1
L1	-	2933.86	2802.09		
L2	-	3268.72	878		
L3	-	5729.72	3184.73		
L4	-	5630.32	1201.24		
S1	1000x1000	8369.79	3118.72	M4	
S2	1000x1000	8330.57	693.85	M3	

Comparing both tables, it is noticeable that more dimensions improve the accuracy of interpreting element dimensions. This is because detailed information is provided for each one. However, although the location dimensions are clearly indicated, there are errors in some positions, suggesting limitations in ChatGPT's ability to interpret overlapping dimensions in complex layouts correctly.

A particular case is the location of workers, which, despite being indicated in the layout with dimensions, ChatGPT tends to misinterpret. This problem recurred even after adjusting the display of machine locations. The exception was when there were only two machines, concluding that overlapping dimensions in more complex layouts make accurate interpretation difficult. This suggests that ChatGPT may become confused when processing multiple close or overlapping dimensions. Thus, an alternative technique was implemented to address this challenge. This consisted of generating, using the software used to create the layout, a table that included the name of each element, its location on the X and Y axes, and its dimensions.

This table, shown in Figure 6, was incorporated into the document below the layout, allowing for a cleaner and more organized design.

TAG	X LOC	Y LOC	SIZE
F-F1	500	500	1000x1000
M-M1	2032	5491	2709x1079
M-M2	3553	1444	3279x1060
M-M3	7739	6228	2586x810
M-M4	8759	652	2499x1048
O-O1	3370	5004	
O-O2	5289	2900	
O-O3	8989	5663	
O-O4	10325	2237	
S-S1	12833	5926	725.33x696.32
S-S2	12985	469	900x900

Figure 6. Image of the table generated in the CAD software.

In addition, visual adjustments were made to improve the layout's clarity: the source was represented in red, machines in green, sinks in purple, and workers were identified with blue circles. The name of each element was circled next to the corresponding object, making it easier to identify. The lines connecting the elements were also maintained, indicating that the upper right corner corresponds to the output and the lower left corner to the input.

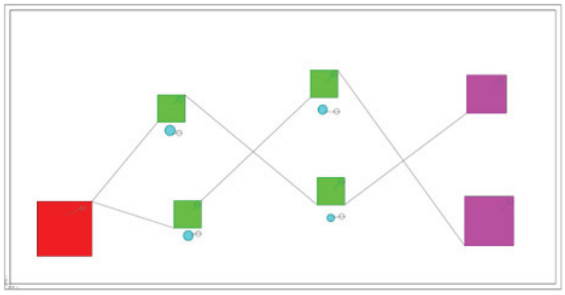

Figure 7. New layout proposed for interpretation with ChatGPT.

Figure 7 shows a visual representation of the proposed layout, where dimensions and scalability are irrelevant, as these data will be obtained directly from the table attached to the document. This approach improves the accuracy of data interpretation and optimizes the code generation process, reducing errors associated with overlapping dimensions and facilitating the creation of more complex models.

Methodology

This section outlines the methodology developed to guide ChatGPT in generating FlexSim code from industrial layout images. The process involves extracting relevant information from a PDF file, interpreting the data, and generating code to create simulation models in FlexSim. The methodology was implemented using ChatGPT and FlexSim (version 23.0.1, 64-bit, educational license) to automate the creation of simulation models while minimizing manual adjustments. The steps are as follows:

a) Data Extraction from the File:

Data extraction involves manually reviewing the PDF layout with a resolution ≥ 300 DPI, to identify key elements-Source, Machine, Operator, Sink-by extracting coordinate pairs (x, y in mm), dimensions (e.g., 1000x1000 mm), and flow connections via directional arrows. For example, a layout specifies:

1. Source (F1): Located at coordinates (500, 500).
2. Machine (M1): Located at coordinates (2032, 5491) with dimensions 2709 x 1079.

Connections between elements are also identified, typically represented by arrows or lines in the diagram. These connections specify the flow of entities (e.g., products, materials) through the system, with the upper right corner representing the output and the lower left corner representing the input.

b) Interpretation of Extracted Data:

The extracted data is interpreted to generate the corresponding FlexSim code. The coordinates of each element are translated into positions within the FlexSim environment. For example, the coordinates (500, 500) for the Source (F1) are mapped to a specific location in the simulation workspace. If necessary, scaling or transformation is applied to ensure the layout fits within the FlexSim environment.
The dimensions of each object (e.g., 2709 x 1079 for Machine M1) are used to define their size in the simulation. Connections between elements are represented in the code using the objectconnect function, which links one object's output to another's input based on the flow specified in the layout.

c) Code Generation from Extracted Data:

The extracted and interpreted data are used to generate FlexSim code through the following steps:

1. Object Creation: For each element (e.g., Source, Machine, Operator, Sink), an object is instantiated in FlexSim using the createinstance function.
2. Coordinate and Size Assignment: The set Location method positions each object, and the dimensions are assigned to define their size.
3. Connection Configuration: The objectconnect method links objects based on the flow specified in the layout.

This process ensures that the simulation model accurately reflects the industrial layout, requiring minimal manual adjustments.

Instruction provided to ChatGPT

The following instruction was provided to ChatGPT to guide the code generation process:

"The attached plan shows the location of the source, machines, operators, and sinks. Use the coordinates specified in the

plan to position each element. The source is represented in red, machines in green, sinks in purple, and operators circled in blue. The lines indicate the connection between the elements, with the upper right corner representing the output and the lower left corner representing the input."

ChatGPT processes this instruction by interpreting the layout and generating the corresponding FlexSim code. The process involves iterative refinement, where the initial code is reviewed and adjusted based on user feedback to ensure accuracy and alignment with the layout.

Results and validation

The combination of the instruction and the coding methodology significantly improved the accuracy and efficiency of the generated code, as demonstrated in Figure 8. Figure 8 illustrates the final simulation model created in FlexSim, showing the accurate placement of elements and their connections based on the industrial layout. The generated code was compared with the original layout to validate the results, and any discrepancies were manually corrected. This iterative process ensured that the simulation model accurately reflected the intended system.

Implementation in FlexSim

The code generated by ChatGPT was implemented in FlexSim (version 23.0.1, 64-bit, educational license). Upon starting a new model, an initial window was displayed (Figure 9), in which the distance units were set to millimeters while the other units were left at their default settings.

After configuring the units, the main window for model creation opened. In the Debug tab, the Script Console option is displayed, as shown in Figures 10 and 11.

```
// Create Source F1 with center position adjustment
Object F1 = createinstance(library().find("?Source"), model());
F1.setLocation(1000, 1000, 0);  // Centered coordinates of F1
F1.name = "F-F1";
F1.size.x = 1000;
F1.size.y = 1000;

// Create Machines M1 to M4 with center position adjustment
Object M1 = createinstance(library().find("?Processor"), model());
M1.setLocation(3386.5, 6029.5, 0);  // Centered coordinates of M1
M1.name = "M-M1";
M1.size.x = 2709;
M1.size.y = 1079;

Object M2 = createinstance(library().find("?Processor"), model());
M2.setLocation(5192.5, 1974, 0);  // Centered coordinates of M2
M2.name = "M-M2";
M2.size.x = 3279;
M2.size.y = 1060;

Object M3 = createinstance(library().find("?Processor"), model());
M3.setLocation(9032, 6628, 0);  // Centered coordinates of M3
M3.name = "M-M3";
M3.size.x = 2586;
M3.size.y = 810;

Object M4 = createinstance(library().find("?Processor"), model());
M4.setLocation(10008.5, 1176, 0);  // Centered coordinates of M4
M4.name = "M-M4";
M4.size.x = 2499;
M4.size.y = 1048;

// Create Operators O1 to O4 with corrected positions
Object O1 = Object.create("Operator");
O1.setLocation(3370, 5004, 0);  // Correct coordinates of O1
O1.name = "O-O1";

Object O2 = Object.create("Operator");
O2.setLocation(5289, 2900, 0);  // Correct coordinates of O2
O2.name = "O-O2";
```

Figure 8. Creating the code generated by Chatgpt.

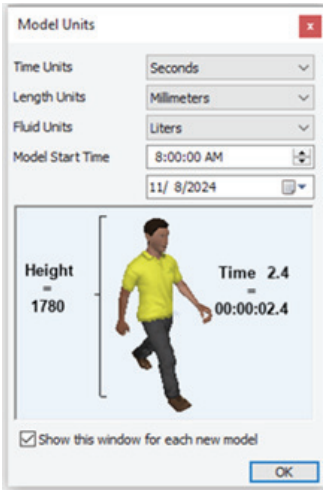

Figure 9. Pop-up window when opening a new FlexSim model.

https://doi.org/10.22201/fi.25940732e.2026.27.1.004

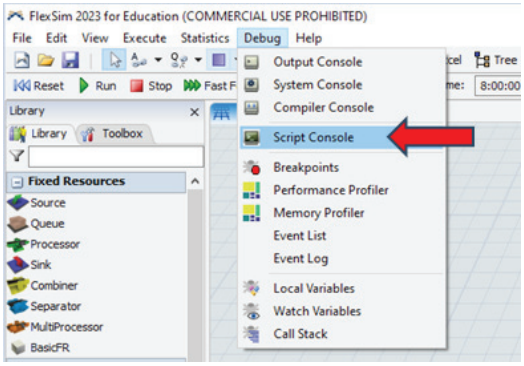

Figure 10. Option within FlexSim to open Script Console.

```
// Create Source F1 with center position adjustment
Object F1 = createinstance(library().find("?Source"), model());
F1.setLocation(1000, 1000, 0);  // Centered coordinates of F1
F1.name = "F-F1";
F1.size.x = 1000;
F1.size.y = 1000;

// Create Machines M1 to M4 with center position adjustment
Object M1 = createinstance(library().find("?Processor"), model());
M1.setLocation(3386.5, 6029.5, 0);  // Centered coordinates of M1
M1.name = "M-M1";
M1.size.x = 2709;
M1.size.y = 1079;

Object M2 = createinstance(library().find("?Processor"), model());
M2.setLocation(5192.5, 1974, 0);  // Centered coordinates of M2
M2.name = "M-M2";
M2.size.x = 3279;
M2.size.y = 1060;

Object M3 = createinstance(library().find("?Processor"), model());
M3.setLocation(9032, 6628, 0);  // Centered coordinates of M3
M3.name = "M-M3";
M3.size.x = 2586;
M3.size.y = 810;

Object M4 = createinstance(library().find("?Processor"), model());
M4.setLocation(10008.5, 1176, 0);  // Centered coordinates of M4
M4.name = "M-M4";
M4.size.x = 2499;
M4.size.y = 1048;

// Create Operators O1 to O4 with corrected positions
Object O1 = Object.create("Operator");
O1.setLocation(3370, 5004, 0);  // Correct coordinates of O1
O1.name = "O-O1";

Object O2 = Object.create("Operator");
O2.setLocation(5289, 2900, 0);  // Correct coordinates of O2
O2.name = "O-O2";

Object O3 = Object.create("Operator");
O3.setLocation(8989, 5663, 0);  // Correct coordinates of O3
O3.name = "O-O3";
```

Figure 11. ChatGPT generated code entered into FlexSim Script Console.

When running the script, objects were automatically generated within the modeling space, centering themselves in the main window while displaying the creation process in real-time. To verify the accuracy of the generated layout, the original design used as reference for ChatGPT was imported into the FlexSim model. This direct comparison confirmed that the positioning and arrangement of all objects precisely matched the expected configuration, as clearly demonstrated in Figure 12.

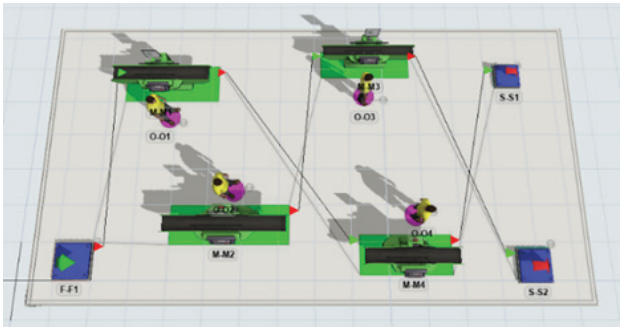

Figure 12. FlexSim environment of the created and connected elements obtained from the ChatGPT code.

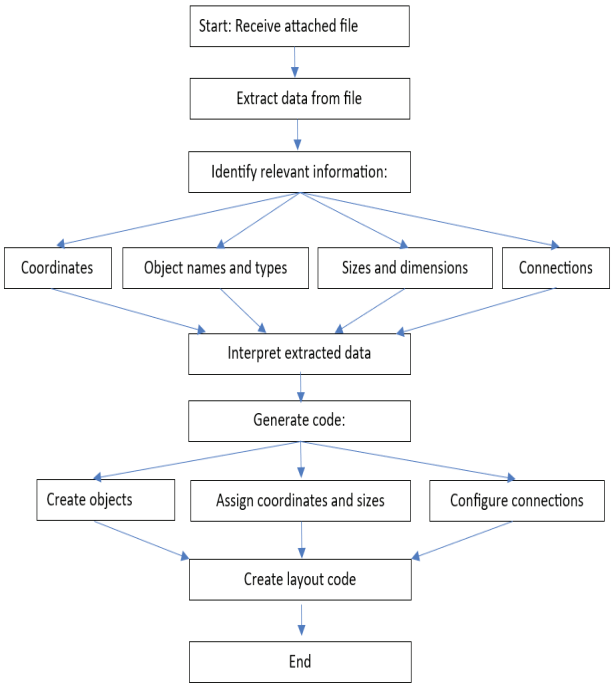

Figure 13. Flowchart: Attachment Receiving and Processing Process.

Conclusions

This study demonstrates a novel approach to automatically generating simulation models in FlexSim using artificial intelligence tools like ChatGPT. By extracting and interpreting data from PDF technical drawings, the methodology partially automates the creation of layouts with multiple elements, including sources, machines, operators, and sinks. The transformation of static information into executable code highlights ChatGPT's ability to interpret and structure relevant data, as illustrated in Figure 13. This approach simplifies the modeling process and offers a valuable tool for optimizing layout construction in simulation environments, reducing time and effort while improving accuracy.

While the differences between automated and manual approaches are less pronounced in simpler layouts with fewer elements, the proposed methodology becomes increasingly relevant for complex layouts. In such cases, the volume of elements and connections significantly increases the difficulty of manual configuration. Automation enhances design accuracy and clarity and reduces the time and effort required for large-scale projects. This scalability offers a strategic advantage in advanced simulations, enabling users to focus on analysis and optimization rather than manual configuration.

This research highlights the potential of AI tools like ChatGPT to streamline simulation model development while identifying key areas for future exploration. For instance, improving ChatGPT's understanding of dynamic semantics and integrating multimodal AI models could enhance its ability to generate more accurate and context-aware code. Additionally, future work will evaluate the effectiveness of this approach in other simulation platforms and industrial applications, such as supply chain optimization and smart manufacturing. Exploring the integration of AI tools with real-time data feeds and IoT systems could further enhance their applicability in dynamic and complex environments.

The validation of AI-generated results is critical to ensuring their reliability and practical applicability in real-world projects. Simulation experts can address complex problems more efficiently by adopting tools like ChatGPT, reducing the time and effort required for model creation. This work establishes a solid foundation for future research, paving the way for integrating AI tools into simulation workflows. As AI continues to evolve, its role in enhancing simulation accuracy, scalability, and accessibility will become increasingly important, driving innovation in industrial engineering, logistics, and smart manufacturing.

Integrating AI tools like ChatGPT into simulation modeling represents a significant step forward in automating complex and time-consuming tasks. By reducing manual effort and improving accuracy, this approach can transform how simulation models are created and optimized, enabling experts to tackle increasingly complex challenges more efficiently. As AI technologies continue to advance, their application in simulation will unlock new possibilities for innovation and problem-solving across a wide range of industries.

However, this time reduction may impact the modeler's knowledge of the system. Over-reliance on AI-generated code could limit the deep understanding gained through manual configuration, potentially reducing the ability to derive nuanced insights into system performance, particularly for complex systems requiring detailed analysis or manual optimization. Despite this, the accuracy of AI-generated models allows users to focus on experimentation and optimization rather than repetitive coding tasks. Nonetheless, there remains a risk of misinterpretation when generating layouts automatically, as ChatGPT may misread images due to low resolution, overlapping dimensions, or ambiguous element labels, leading to errors in the digital representation within FlexSim (e.g., incorrect object placements or connections). Consequently, users must always verify the generated layout using FlexSim's 3D visualization or Script Console to ensure it matches the original design. This validation is critical, especially for complex layouts where AI struggles with dense or overlapping information, as errors could propagate to later stages, affecting performance analysis. Iterative feedback to ChatGPT can improve accuracy, but the user's system knowledge remains essential to identify discrepancies and ensure model reliability, reinforcing that AI complements, rather than replaces, human expertise.

References

Averill, M. L. (2015). *Simulation modeling and analysis*. 1ª ed. New York: McGraw-Hill Education.

Banks J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (2010). *Discrete--Event system simulation*. 4th ed. Prentice-Hall.

Bilgram, V., & Laarmann, F. (2023). Accelerating innovation with generative AI: AI-augmented digital prototyping and innovation methods. *IEEE Engineering Management Review,* 51(2), 18-25. https://doi.org/10.1109/EMR.2023.3272799

Brynjolfsson, E., Li, D., & Raymond, L. R. (2023). Generative AI at work. *Journal of Economic Perspectives,* 37(4), 1-22. https://doi.org/10.1093/qje/qjae044

Flexsim. (2024). Fliexsim simulation software. Online. Retrieved on https://www.flexsim.com

Flexsim. Flexsim.com. Online. Retrieved September 20, 2024 on https://docs.flexsim.com/en/23.0/Reference/Tools/ScriptConsole/ScriptConsole.html

Flexsim. FlexScript Class Reference. Online. Retrieved on https://docs.flexsim.com/en/23.0/Reference/CodingInFlexSim/FlexScriptAPIReference/FlexScriptClassReference.html

García-Jacobo, F. & Romero-Guerrero, J. (2020). Diseño de un modelo de simulación, utilizando un software de eventos discretos, en una línea de producción de tejido industrial. *Revista Internacional de Investigación e Innovación Tecnológica,* Retrieved on https://riiit.com.mx/apps/site/idem.php?module=Catalog&action=ViewItem&id=6216&item_id=85369

Garrido, J. (2009). *Introduction to Flexsim. In principles of simulation modeling*. Spring, 31-42. Retrieved on https://openai.com/chatgpt

Gonzalo-Brizuela, R., & Garrido-Merchan, E. C. (2023). ChatGPT is not all you need. A state of the art review of large generative

AI models. *Computer Science.* Cornell University. https://doi.org/10.48550/arXiv.2301.04655

Hamilton, A. (2024). Artificial intelligence and healthcare simulation: The shifting landscape of medical education. *Cureus,* 16(5), e59747. https://doi.org/10.7759/cureus.59747

Herrera-Ortiz, J., Peña-Avilés, J., Herrera-Valdivieso, M., & Moreno-Morán, D. (2024). La inteligencia artificial y su impacto en la comunicación: recorrido y perspectivas. *Revista de Estudios Interdisciplinarios en Ciencias Sociales,* 26(1), 278-296. https://doi.org/10.36390/telos261.18

Megahed, F. M., Chen, Y. J., Ferris, J. A., Knoth, S., & Jones-Farmer, L. A. (2023). How generative AI models such as ChatGPT can be (mis)used in SPC practice, education, and research? An exploratory study. *Quality Engineering,* 36(2), 287-315. https://doi.org/10.1080/08982112.2023.2206479

Tian, H., Lu, W., On, T., Tang, X., Cheung, S., Klein, J., & Bissyandé, T. F. (2023). Is ChatGPT the ultimate programming assistant-how far is it? *Computer Science,* Cornell University. https://doi.org/10.48550/arxiv.2304.11938

Hou, M., & Fidopiastis, C. (2016). A generic framework of intelligent adaptive learning systems: From learning effectiveness to training transfer. *Theoretical Issues In Ergonomics Science*, 18(2), 167-183. https://doi.org/10.1080/1463922x.2016.1166405

Jackson, I., Saenz, M. J., & Ivanov, D. (2023). From natural language to simulations: Applying AI to automate simulation modelling of logistics systems. *International Journal of Production Research*, 62(4), 1434-1457. https://doi.org/10.1080/00207543.2023.2276811

Kobiella, C., Flores, Y., Waltenberger, F., Draxler, F., & Schmidt, A. (2024). If the machine is as good as me, then what use am I?- How the use of ChatGPT changes young professionals. Perception of productivity and accomplishment. *CHI '24: Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems,* 1018, 1-16. https://doi.org/10.1145/3613904.3641964

Wan, J., Li, X., Dai, H., Kusiak, A., Martínez-García, M., & Li, D. (2021). Artificial intelligence-driven customized manufacturing factory: Key technologies, applications, and challenges. *Proceedings of the IEEE,* 109(4), 377-398. https://doi.org/10.1109/jproc.2020.3034808

Ma, W., Liu, S., Wang, W., Hu, Q., Zhang, C., Liu, Y., Nie, L., & Liu, Y. (2023). ChatGPT: understanding code Syntax and semantics. *Computer Science,* Cornell University, https://doi.org/10.48550/arxiv.2305.12138

Majeed, A., & Hwang, S. O. (2024). Making large language models more reliable and beneficial: Taking ChatGPT as a case study. *Computer*, 57(3), 101-106. https://doi.org/10.1109/mc.2023.3327028

Combrink, H., Brokensha, S., Van, C., Matlhoko, K., Merwe, S., & Merwe, T. (2023). Stepping up with ChatGPT: A comprehensive guide for academics. University of the Free State. Online resource.

Muthukrishnan, N., Maleki, F., Ovens, K., Reinhold, C., Forghani, B., & Forghani, R. (2020). Brief history of artificial intelligence. *Neuroimaging Clinics of North America*, 30(4), 393-399. https://doi.org/10.1016/j.nic.2020.07.004

OpenAI. (2023). ChatGPT: Optimizing language models for dialogue. Online. Retrieved on https://openai.com/chatgpt

Sadik, A., Brulin, S. & Olhofer, M. (2024). Coding by Design: GPT-4 empowers agile model driven development. In Proceedings of the 12th International Conference on Model-Based Software and Systems Engineering 1, 146-156. Retrieved on https://doi.org/10.5220/0012356100003645

Wang, X., Chen, G., Qian, G., Gao, P., Yong, X., Wang, Y., Tian, Y., &. Gao, W. (2023). Large-scale multi-modal pre-trained models: A comprehensive survey. *Machine Intelligence Research,* (20), 447-482. https://doi.org/10.48550/arXiv.2302.10035

How to cite:

Romero-Guerrero, J. A., Bautista-Orduña, G. E., Suarez-Luna, J. M., & Arenas-Islas, D. (2026). Creation of discrete event simulation models using artificial intelligence and flexsim. *Ingeniería Investigación y Tecnología,* 27(01), 1-12. https://doi.org/10.22201/fi.25940732e.2026.27.1.004